

RAG Fundamentals Study Notes

1. What is RAG?

Retrieval-Augmented Generation (RAG) combines information retrieval with a Large Language Model. Instead of relying only on what the model learned during training, RAG retrieves relevant information from external documents and provides it to the model before generating an answer.

2. High-Level Architecture

Documents -> Chunking -> Embeddings -> Vector Database -> Retrieval -> Prompt Augmentation -> LLM -> Answer.

3. Embeddings

Embeddings convert text into vectors (lists of numbers). Similar meanings produce similar vectors. Embeddings are the foundation of retrieval systems.

4. Vector Databases

A vector database stores embeddings and allows efficient similarity search. Examples include FAISS, Chroma, Pinecone, and Weaviate.

5. Similarity Search

The user's query is converted into an embedding. The system finds the most similar document chunks using metrics such as cosine similarity, dot product, or Euclidean distance.

6. Chunking

Large documents are split into smaller chunks. Chunk size and overlap significantly affect retrieval quality.

7. Retrieval

The system retrieves the top-k most relevant chunks. Retrieval quality determines the quality of the final answer.

8. Prompt Augmentation

Retrieved chunks are inserted into the prompt as context before sending the prompt to the language model.

9. Generation

The language model generates an answer using both the user's question and the retrieved context.

10. Topics to Master Before Building RAG

Tokenization, Embeddings, Cosine Similarity, Vector Search, FAISS, Chunking Strategies, Retrieval, Prompt Engineering, and LLM Fundamentals.

Suggested Learning Order

1. Embeddings and Vector Spaces
2. Cosine Similarity
3. Vector Databases (FAISS)
4. Chunking Strategies
5. Retrieval and Top-K Search
6. Prompt Augmentation
7. End-to-End RAG Implementation